Personal Research Agent: Architecture Design

Oct 22, 2026 · @tj

Overview and key decisions

The Personal Research Agent is a single Python process that runs a plan, research, write, verify pipeline around an LLM, with tools for search and page fetching and a local database for notes. It implements the Requirements Specification doc.

Design goals

- **Grounded**: every claim in the report comes from a stored note with a source URL.
- **Bounded**: step, token, and time budgets stop every loop.
- **Simple**: no server, no agent framework; one command runs it.
- **Swappable**: the model, search provider, and storage sit behind small interfaces.
- **Observable**: every step is logged with its tool, duration, and token count.

Key decisions

Decision	Choice	Why
Agent style	Fixed pipeline; open tool-use loop only inside the research step	Predictable cost and easier to test than a free-roaming agent, while still teaching tool use
Framework	None; call the model provider's SDK directly	You learn the agent loop itself and keep dependencies small
Interface	Command line first	Smallest surface; a web UI can wrap the same orchestrator later
Storage	One SQLite file	No server, queryable, easy to inspect
Writer input	Notes only, never raw pages	Keeps context small and makes every claim traceable
Language	Python 3.11 or newer	Strong library support for HTTP, PDF, and LLM SDKs

System architecture

The orchestrator is the only component that decides what happens next. Every tool is a plain function behind an interface, so the model, the search provider, and storage can each be swapped without touching the loop.

One orchestrator runs four steps; tools and storage sit behind interfaces



system architecture · orchestrator, 4 tools, 3 external systems

The command line starts a run; the orchestrator calls the planner, researcher, writer, and verifier in turn, and each reaches the outside world only through a tool.

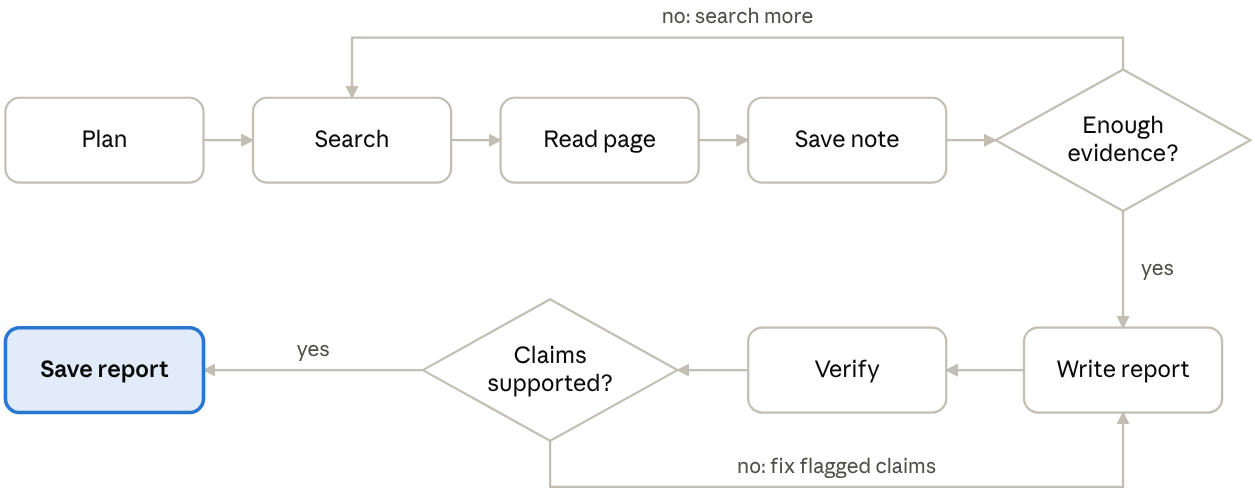
Agent loop and workflow

A run has five steps, and only the research step is an open-ended loop. Budgets bound every loop.

1. **Plan**: the planner turns the question into 3-6 sub-questions (FR-3) and optionally waits for approval (FR-4).
2. **Research**: for each sub-question the model calls `web_search`, `fetch_page`, and `save_note` in a loop until it calls `finish_research` or a budget runs out.
3. **Select**: notes are ranked by relevance and credibility; weak and duplicate ones are dropped.

4. **Write**: the writer sees only the plan and the notes, and produces a report with numbered citations.
5. **Verify**: the verifier checks each claim against its cited note; unsupported claims are rewritten or removed.

Both loops end on evidence: enough sources, then supported claims



agent loop · 2 decisions, 2 retry loops

The search loop repeats for each sub-question; the write loop repeats until the verifier finds no unsupported claims or a budget stops it.

Stop conditions: the model signals the sub-question is done, the step, token, or time budget is hit, or the source limit is reached. On a budget stop the agent still writes a report from the notes it has and labels it partial.

Tools and interfaces

The model can call four tools; the final report is written by a separate model call, not a tool.

Tool	Input	Output	Notes
`web_search`	query, max results	list of title, URL, snippet	Wraps the chosen search provider
`fetch_page`	URL	clean text, title, content type, fetch time, or an error	Handles HTML and PDF; timeout, size cap, robots.txt respected; cached by URL

Tool	Input	Output	Notes
`save_note`	source id, key facts, short evidence quote	note id	Evidence quotes stay under 25 words (FR-8)
`finish_research`	sub-question id, one-line status	done signal	Ends the loop for that sub-question

Two small interfaces keep the vendors replaceable:

```
class LLMClient(Protocol):
    def complete(self, system: str, messages: list[dict],
                 tools: list[dict] | None = None) -> LLMResponse: ...

class SearchProvider(Protocol):
    def search(self, query: str, max_results: int) -> list[SearchResult]: ...
```

Data model, state, and memory

One SQLite file holds everything a run produces, so a session can be inspected, resumed, or reused later.

Table	Key fields	Purpose
sessions	id, question, depth, status, budget used, created at	One row per run
sub_questions	id, session id, text, status	The plan
sources	id, session id, URL, title, fetched at, content hash, status, reason	Every page tried, including failures and skips
notes	id, source id, sub-question id, facts, evidence quote, relevance score	The only input the writer sees
reports	id, session id, Markdown, claims checked, created at	Final output and verification result

Memory

- **Short-term**: the message history of one research loop, cleared between sub-questions so context stays small.
- **Long-term**: notes and sources in SQLite, reused for follow-up questions (FR-18) and to avoid refetching the same URL.

Tech stack and project structure

Layer	Choice	Reason
Language	Python 3.11+	Matches NFR-11
LLM	Anthropic Python SDK, behind `LLMClient`	Native tool use; swappable
Search	One provider with a free tier, behind `SearchProvider`	Still an open question in the requirements
HTTP	httpx	Timeouts, retries, async-ready
HTML extraction	trafilatura	Pulls main article text from pages
PDF extraction	pypdf	Simple text extraction
Schemas	pydantic	Typed tool inputs and outputs
Storage	sqlite3 from the standard library	No extra dependency
CLI and progress	typer and rich	Clean commands and live progress
Tests	pytest	Fakes and fixtures for offline runs

```
pra/
  cli.py            command-line entry point
  orchestrator.py   plan, research, write, verify pipeline
  budget.py         step, token, and time limits
  llm/              LLMClient interface and Anthropic implementation
  tools/            web_search.py, fetch_page.py, notes.py
  prompts/          planner.md, researcher.md, writer.md, verifier.md
  store.py          SQLite access
  models.py         typed schemas for sessions, sources, notes
tests/
  fakes.py          fake LLM and fake search for offline tests
  fixtures/         saved pages and search results
  eval/             evaluation questions and scoring script
reports/            generated Markdown reports
```

Prompts, error handling, guardrails, and cost control

Prompts. Each step has its own prompt file, so you can tune one without breaking the others.

- **Planner**: output 3-6 sub-questions as JSON; cover definitions, current state, and disagreements.
- **Researcher**: use tools only; save a note for every useful fact; stop when evidence is enough.
- **Writer**: use only the provided notes; cite every factual claim as [n]; say when evidence is thin.
- **Verifier**: for each claim, answer supported, partly supported, or unsupported, using only the cited note.

Error handling

Failure	Handling
Search API error or empty results	Retry up to 3 times with backoff, rephrase the query once, then log a coverage gap
Page fetch fails, times out, or is blocked	Mark the source failed with a reason and move on
Page is not readable text	Skip with a reason

Failure	Handling
Malformed tool call from the model	Return the error to the model once, then end that sub-question
Budget exhausted	Stop loops, write a partial report from existing notes, say so in the report
Claim with no supporting note	Rewrite or remove it and flag it in the report

Guardrails

- Fetched text is wrapped as quoted data in the prompt and never treated as instructions (NFR-6).
- The model can only call the four listed tools; none has side effects outside the local machine.
- Page text is capped at a configurable length before it reaches the model.
- API keys come from environment variables and are redacted from logs.

Cost control

- A budget object tracks steps, tokens, and seconds and is checked on every loop iteration.
- Use a smaller, cheaper model for note extraction and verification, and the stronger model for planning and writing.
- Cache fetched pages by URL hash so reruns and follow-ups do not refetch.

Testing, deployment, and build order

Testing

Level	What is checked	How
Unit	Each tool, the budget, the store	pytest with saved page fixtures
Pipeline	Full run with no network	Fake LLM and fake search returning scripted results
Evaluation	Claim accuracy and citation quality (NFR-3, AC-1 to AC-8)	A set of 20 test questions, scored by hand against the cited sources

Deployment. Local only: install with `pip install -e .`, set the API keys as environment variables, and run `pra "your question"`. Reports land in `reports/`.

Build order

1. Project skeleton, config, logging, and the `LLMClient` wrapper.
2. `fetch_page` with HTML and PDF extraction, tested on saved pages.
3. `web_search` behind `SearchProvider`.
4. Research loop with `save_note` and the budget object.
5. Planner and writer prompts; first end-to-end cited report (the MVP).
6. Verifier pass, source ranking, and saved sessions.
7. Evaluation set, then follow-up questions and a simple web UI.